

Aufgaben als Prüfungsvorbereitung

Inhaltsverzeichnis

1. Übungsmodelle/Übungsszenarien	1
1.1. Szenario 1: Semester-Planung	2
1.2. Szenario 2: Stromkunden-Verwaltung	2
1.3. Szenario 3: World of Codecraft	2
2. Programmieraufgaben	3
2.1. Aufgabe 1: Aufzählungen	3
2.2. Aufgabe 2: "Gleich ist nicht gleich gleich"	3
3. Quizfragen (10)	4

1. Übungsmodelle/Übungsszenarien

Aufgabe:

Anhand der, durch Kunden, natürlich-sprachlich formulierten Anwendungsszenarien sollen Java Klassenmodelle erstellt werden. Grundsätzlich können/sollen dabei möglichst folgende *Phasen* durchlaufen werden:

1. **Verstehen der "Fachlichkeit"** durch Analyse der Anforderungen
2. **Festlegung** z.B. von Klassen-, Feld- oder Methodennamen
3. **Grafische Umsetzung** des Modells, empfohlen wird UML2, andere Notationen sind aber erlaubt. Sie müssen aber vom Kunden verstanden werden können (z.B. mittels Legende)
4. **Technische Umsetzung** in konkrete Pakete, Klassen, Felder und/oder Methoden (Operationen)
5. **Testen** (Beweisführung) des technischen Modells durch ein oder mehrere Unit-Tests, etwa durch konkrete Instanziierung der Modellklassen

Wurzel-**Package** für die Modellklassen, ggf. pro Szenario noch ein zusätzliches Package zur Vermeidung von Namenskonflikten und zur besseren Strukturierung:

```
src/main/java/de/dhbw/exam/*.java
```

Zugehöriger Unit-Tests:

```
src/test/java/de/dhbw/exam/ScenarioTests.java
```

1.1. Szenario 1: Semester-Planung

Der Kunde **DHBW** beauftragt die Entwicklung einer Software zur besseren Planung von Lehrveranstaltungen, die im Rahmen eines Studiums durchgeführt werden.

Dazu sollen einzelne **Inhalte**, die Gegenstand einer LV sind, als eine Art Bausteine zunächst erstellt werden können. Diese sollen dann für die **zeitliche Planung** von Lehrveranstaltungen herangezogen werden können.

Wünschenswert ist auch, dass die Lehrinhalte **kategorisiert** werden können und je Baustein eine **Dauer** angegeben werden kann.

Mögliche ...

- Klassen
- Felder
- Methoden

1.2. Szenario 2: Stromkunden-Verwaltung

Der Kunde **DB Energie** entwickelt eine Software zur Verwaltung ihrer Stromkunden.

Dazu haben die **Kunden** jeweils ein oder mehrere **Accounts**. Für jeder Account kann es höchstens einen **Vertrag** geben, in dem der **Tarif** der Stromlieferung definiert ist.

Wichtig ist auch, dass für jeden Kunden der **Gesamtstromverbrauch** eines Jahres gespeichert werden kann

Mögliche ...

- Klassen
- Felder
- Methoden

1.3. Szenario 3: World of Codecraft

Der Kunde **CodeSteam** möchte ein MMORPG im Internet anbieten.

Im Spiel gibt es eine *feste Menge* (3) von **Quests**, die von **Spielern** bzw, deren **Avataren** gespielt werden können. Gleichzeitig können Spieler andererseits mehrere Quests beginnen. Jedem Spieler steht dabei aber *nur eine* **Waffe** zur Verfügung.

Zu beachten ist, dass *laufende bzw. bereits begonnene Quests* durchaus auch abgebrochen werden können.

Mögliche ...

- Klassen
- Felder
- Methoden

2. Programmieraufgaben

Wurzel-**Package**, nutzbar für mögliche Implementierungen/Klassen, die im Rahmen der Aufgabe erstellt werden können:

```
src/main/java/de/dhbw/exam/*.java
```

Zugehöriger **Unit-Tests**:

```
src/test/java/de/dhbw/exam/ProgrammingTests.java
```

2.1. Aufgabe 1: Aufzählungen

Gegeben sei die folgende **Aufzählung**. Implementiere zwei beispielhafte **switch** Anweisung für die Elemente dieses **enum**s, einmal in "alter" Form und einmal als "enhanced switch". Nutze dazu einfach einen **Unit-Tests**:

```
public enum Spielkarte {  
    KREUZ, PIK, HERZ, CARO;  
}
```

2.2. Aufgabe 2: "Gleich ist nicht gleich gleich"

Übung zur Klasse **Object** bzgl. des **Gleichheitskonzeptes** in Java:

- Erstellen Sie eine Klasse **Password**
- mit einem Feld **String value** und
- einem **Konstruktor**, über den das Passwort angegeben wird
- Testen Sie die Gleichheit der folgenden zwei Instanzen mit gleichem Passwortwert in einem Unit-Test

```
Password p1 = new Password("x84jdf754hf6");  
Password p2 = new Password("x84jdf754hf6");
```

- Überschreiben Sie in **Password** die Object-Methoden **equals()** und **hashCode()** unter Nutzung des Feldes **value** und möglichst mittels Code-Generierung, und zum Schluss

- Testen Sie erneut, ob die obigen Instanzen gleich sind

3. Quizfragen (10)

Wurzel-**Package**, nutzbar für mögliche Implementierungen/Klassen, die im Rahmen der Aufgabe erstellt werden können:

```
src/main/java/de/dhbw/exam/*.java
```

Zugehöriger **Unit-Tests**:

```
src/test/java/de/dhbw/exam/QuizTests.java
```

Frage 1: Welche der folgenden **Listentypen** wäre am passendsten zur Speicherung von Listenelementen geeignet, wenn ein Listenelement aus einem Schlüssel und einem Wert besteht und deren Einträge natürlich sortiert werden sollen?

- A. Collection
- B. List
- C. Set
- D. TreeMap
- E. Vector

Frage 2: Nenne zwei **Eigenschaften** der Klasse/Implementierung **LinkedList** aus dem *Collections* Framework?

Frage 3: Was bedeutet **Überladen** (von Methoden)?

Frage 4: Setzen Sie die erforderlichen **Schlüsselworte** im folgenden kleinen Klassenmodell ein!

```
public interface Executable {}  
public abstract class Task _____ Executable {}  
public class Batchtask _____ Task {}
```

Frage 5: Was ist der Unterschied zwischen **impliziter** und **expliziter Umwandlung** von primitiven Datentypen? Nennen Sie je ein Beispiel.

Frage 6: Welcher **Beziehungstyp** mit welcher **Kardinalität** ist hier implementiert?

```
public class Building {  
    private List<Room> rooms;  
}
```

Frage 7: Was ist eine **generische** Klasse?

Frage 8: Was ist ein **funktionales Interface** und nennen sie mindestens 2 aus dem Bereich der Lambda's?

Frage 9: Was ist die **Bedeutung** des Ausdrucks **checked exception**?

Frage 10: Welche der drei Aussagen trifft bzgl. des Schlüsselwortes **static** zu?

(der Code-Ausschnitt unten dient als Ansichtsbeispiel)

- A. Der Aufruf einer statischen Methode kann nur über eine Instanz der umgebenden Klasse erfolgen
- B. Der Wert einer statischen Klassenvariable kann nicht geändert werden
- C. Der Aufruf einer statischen Methode erfordert keine Instanz der umgebenden Klasse

```
public class DateConverter {  
    public static String TIME_ZONE = "UTC";  
    public static Date parse(String dateAsString) { ... }  
}
```